

Training Coding Agents

Executable tasks, interaction data, and learning objectives

CMSC 848N · W8-L1 · October 20, 2026

What should the model learn from a repair?

A useful run may contain a failed edit, a test failure, and a recovery.

The final patch tells us what changed. The interaction record tells us how the agent got there.

Our choices: which runs to keep, which tokens to predict, and which outcomes to reward.

A training example needs a task, an interaction record, and an outcome label.

A task can produce many trajectories

$$x = (q, c_{\text{init}}, e, \mathcal{T}, h, B)$$

$$\tau = (o_0, u_0, o_1, u_1, \dots, o_T), \quad \tau \sim P_{\theta, h}(\cdot \mid x)$$

Task: issue q , initial code state, environment e , tests T , harness h , budget B .

Trajectory: observations o and complete model-generated messages u .

Keep task identity separate from run identity.

A real executable task record

SWE-smith · OAuthlib · selected fields from one released row

Field	Recorded value
instance_id	oauthlib...combine_file__09vlzwgc
image_name	...oauthlib_1776_oauthlib.1fd52536
FAIL_TO_PASS	18 test identifiers
PASS_TO_PASS	655 test identifiers

The record contains an injected bug patch and an issue description.

The injected patch reverses a scope list

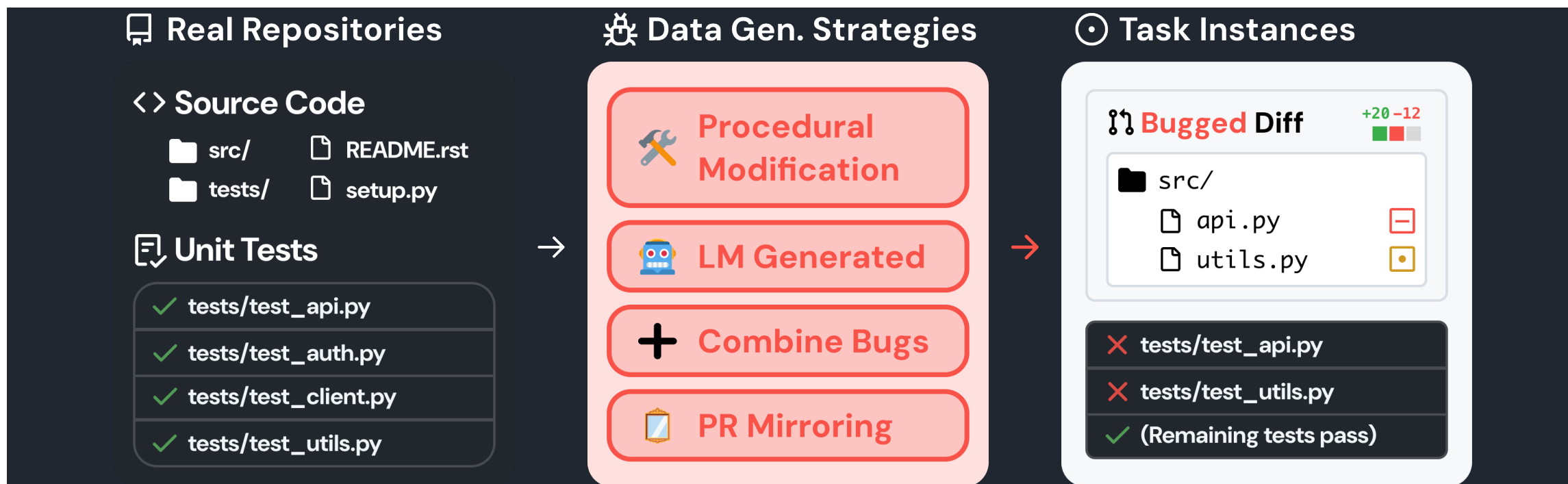
Authentic excerpt: oauthlib/oauth2/rfc6749/utils.py

```
- elif isinstance(scope, (set, tuple, list)):
-     return " ".join([str(s) for s in scope])
+ elif isinstance(scope, (tuple, list)):
+     return " ".join(str(s) for s in reversed(scope))
```

This is one hunk of a combined bug; other functions also change.

SWE-smith starts from a runnable repository

Four strategies produce candidate bugs for test-based validation.



After introducing a bug, execute tests to decide whether the candidate is useful.

The test partitions answer different questions

$$\mathcal{T}_{F2P} = \{t : t(c_b) = 0, t(c_0) = 1\}$$

$$\mathcal{T}_{P2P} = \{t : t(c_b) = 1, t(c_0) = 1\}$$

Subscripts 0 and b denote clean and injected-bug states. A result of 1 means pass.

A repair should fix the observed regressions and preserve working behavior.

A test that already fails in the clean environment cannot certify this injected bug.

Verifier checks on known states

Sanity checks and counterexamples for a repair verifier

Repository state	Expected verifier result
Buggy starting state	Fail
Reference repair applied	Pass
Unrelated source change	Still fail on the targeted behavior
Independent valid repair	Pass if it satisfies the specification

The last two checks probe shortcuts and overly narrow tests.

A verifier can pass the wrong implementation

Weak verifier: source matching

```
grep -q "sorted(values)" impl.py
```

Behavioral probe: course example

```
from impl import normalize
assert normalize([3, 1]) == [1, 3]
assert normalize([]) == []
```

Specification: sort the integers. Can the left check pass without doing that?

SWE-Universe builds verifiers from pull requests

Separate a pull request into test changes and a reference fix.

A builder creates an environment and an evaluation script.

Validate buggy/fixed states; inspect scripts for superficial checks.

The paper reports 807,693 environments across 52,960 repositories.

Task admission checks

A proposed audit checklist for a small research dataset

Check	Failure it can reveal
Pinned clean rebuild	Environment drift
Repeated tests with seeds	Flaky labels
Issue–test agreement	Reward misses the requested behavior
Evaluator access controls	Agent changes its evaluator

Record infrastructure failures separately from incorrect repairs.

A rollout record supports replay and audit

Store ordered messages, tool calls, tool outputs, and the submitted patch.

Bind the run to a task revision, harness commit, model, sampling settings, and budget.

Keep verifier results and raw logs outside the model's claimed success message.

The agent saying “done” is an event in the trajectory.

Successful-run filtering changes the data distribution

$$\mathcal{D}_+ = \{(x, \tau) : V_x(\Delta(\tau)) = 1\}$$

$$p_+(\tau \mid x) = \frac{p_{\text{teacher},h}(\tau \mid x) \mathbf{1}[V_x(\Delta(\tau)) = 1]}{\Pr(V_x = 1 \mid x)}$$

Keep runs whose final patch passes the training verifier.

The conditional distribution still depends on the teacher and harness.

Defined for tasks with nonzero teacher success probability.

Easy tasks can dominate the accepted trajectories

$$\Pr_{\text{kept}}(x) \propto p_{\text{sample}}(x) \Pr(V_x = 1 \mid x)$$

Course example: 10 attempts each; task A succeeds 9 times, task B once.

A supplies nine times as many trajectories. Token weighting also depends on length.

Choose task-level weighting and per-task caps deliberately.

A resolved label still needs an explicit filter

Pinned SWE-smith collector · two separated excerpts

```
traj["resolved"] = is_resolved
traj["traj_id"] = f"{folder}.{generate_hash(str(traj_dir))}"
# ...
for _, traj in results:
    f.write(json.dumps(traj) + "\n")
```

For successful-only SFT, an explicit selection step is still required.

The configuration helps define the loss mask

Pinned SWE-smith 32B configuration · selected fields

```
dataset:  
  _component_: torchtune.datasets.chat_dataset  
  conversation_column: messages  
  conversation_style: openai  
  train_on_input: False  
  packed: False  
seed: 42
```

Inspect token labels after formatting; a configuration flag is only one part of the path.

The token-level supervised objective

$$\mathcal{L}_{\text{SFT}}(\theta) = - \frac{\sum_{n=1}^N \sum_{j=1}^{L_n} m_{n,j} \log \pi_{\theta}(z_{n,j} \mid z_{n,<j})}{\sum_{n=1}^N \sum_{j=1}^{L_n} m_{n,j}}$$

$m_{n,j} = 1$ for selected assistant tokens; otherwise 0

The prefix includes the issue, prior actions, and tool observations.

Masked observations still influence the predicted action through the context.

Assume at least one selected token. This objective uses token weighting.

Removing a failed action can break the example

Illustrative serialized interaction

```
assistant: edit the function at lines 20–28  
observation: SyntaxError; original file restored  
assistant: inspect lines 20–32  
observation: [numbered source excerpt]  
assistant: replace lines 20–29  
observation: edit accepted
```

Preserve the action–observation history needed to explain the recovery.

Online learning changes the rollout distribution

$$P_{\theta,h}(\tau \mid x) = p(o_0 \mid x) \prod_{t=0}^{T-1} \pi_{\theta}(u_t \mid H_t) P_h(o_{t+1} \mid H_t, u_t)$$

$$\nabla_{\theta} J = \mathbb{E} \left[(R(\tau) - b(x)) \sum_t \nabla_{\theta} \log \pi_{\theta}(u_t \mid H_t) \right]$$

H_t is the interaction prefix; u_t is the complete model-generated message.

Assume the environment and fixed harness have no θ -dependent parameters.

The gradient uses generated-message probabilities. Tool outputs condition later actions.

GRPO compares attempts on the same task

$$\tau_1, \dots, \tau_G \sim P_{\theta_{\text{old}}, h}(\cdot \mid x)$$

$$\bar{R} = \frac{1}{G} \sum_i R_i, \quad s_R = \sqrt{\frac{1}{G} \sum_i (R_i - \bar{R})^2}, \quad A_i = \frac{R_i - \bar{R}}{s_R + \delta}$$

Use the same task and evaluation protocol for every member of the group.

$\delta > 0$ handles a zero-variance group in this teaching formulation.

This multi-turn GRPO formulation adapts DeepSeekMath to coding trajectories.

One successful attempt gives a relative signal

Worked example: $G = 4$, rewards $[0, 0, 1, 0]$, negligible δ

Attempt	Reward	Normalized advantage
1	0	$-1 / \sqrt{3} \approx -0.577$
2	0	$-1 / \sqrt{3} \approx -0.577$
3	1	$\sqrt{3} \approx 1.732$
4	0	$-1 / \sqrt{3} \approx -0.577$

Mean = $1/4$; population standard deviation = $\sqrt{3}/4$.

A clipped surrogate for the policy update

$$\rho_{i,j}(\theta) = \frac{\pi_{\theta}(z_{i,j} \mid H_{i,j})}{\pi_{\theta_{\text{old}}}(z_{i,j} \mid H_{i,j})}$$

$$\mathcal{J} = \frac{1}{G} \sum_i \frac{1}{|M_i|} \sum_{j \in M_i} [\min(\rho_{i,j} A_i, \text{clip}(\rho_{i,j}, 1 - \epsilon, 1 + \epsilon) A_i) - \beta K_{i,j}]$$

M_i indexes generated tokens across the interaction. K is a reference-policy KL term.

The old policy generated the batch. The reference policy anchors behavior.

Clipping reduces the incentive for large ratio changes; it gives no hard global bound.

Uniform rewards leave no within-group ranking

$$R_1 = \dots = R_G \implies A_1 = \dots = A_G = 0$$

All failures: the sampled group contains no successful behavior to favor.

All successes: the binary outcome does not distinguish the attempts.

The advantage term vanishes; a KL term may still produce a gradient.

The RL system must preserve the experiment

What each rollout worker must return to the learner

Record	Why it matters
Task, image, harness revision	Environment distribution
Policy, token probabilities, contexts	Reconstruct the importance ratio
Stop reason + resource use	Budget or infrastructure failure
Verifier, patch, execution logs	Reward audit

Reset isolation and masks define the learning problem that the system implements.

A noisy verifier changes the objective

$$\mathbb{E}[\hat{R}] = \alpha + (1 - \alpha - \eta) \Pr(R^* = 1)$$

Course model: α = false-positive rate; η = false-negative rate.

Constant error rates give this expression. Patch-dependent errors can reward shortcuts.

R^* denotes intended correctness; $\hat{R}$ is the observed verifier reward.

SWE-smith results under the reported setup

Reported experiment in the May 2025 paper

Item	Reported setting
Student	Qwen 2.5 Coder Instruct 32B
Training	5,016 expert trajectories; supervised fine-tuning
Evaluation	SWE-bench Verified; SWE-agent; temperature 0
Result	40.2% resolved; 75-step and \$2 proxy-cost caps

This is a historical result, not a claim about the current leaderboard.

Observation prediction is a separate objective choice

Assistant-only SFT selects the model's generated messages.

SWE-Universe's reported mid-training uses no loss mask, including observations.

Its agentic RL stage uses executable task feedback.

State the objective for each training stage separately.

Data separation before rollout collection

Separate tasks by issue and patch identity; keep related variants together.

Use validation tasks for checkpoint selection and untouched final test tasks.

Freeze the harness, budget, and verifier; test repository or temporal transfer.

A new trajectory of an old task is still an old task.

Which records would you admit to SFT?

Four-minute pair exercise · hypothetical collection audit

Run	Audit finding
A: passes	Same task has 29 other retained successes
B: passes	Verifier checks a source-code string
C: fails	Dependency download failed before agent editing
D: passes	A failed edit and recovery are preserved in order

For each: keep, reject, or investigate? State the evidence you need.

Selection, weighting, and supervision

One defensible audit response

Run	Proposed action
A	Keep selectively or reweight at the task level
B	Repair and revalidate the verifier first
C	Classify as infrastructure failure; retry under policy
D	Retain causal history; choose action masks explicitly

Record the policy so that another group can reproduce the training set.

A small controlled study can answer a useful question

Proposed study: what does interaction supervision contribute?

Arm	Training representation
A	Issue + final patch; shared action format
B	Successful interaction; assistant-token loss
C	Same interactions; all-token loss
Evaluation	Held-out tasks; same initial model and evaluation protocol

B versus C isolates the loss mask. A versus B compares demonstration formats.

Next: improve the runtime around the trained model

Read SWE-smith §§2–4 alongside its collector and training configuration.

Read SWE-Universe §§2 and 5 with attention to verifier assumptions.

Next lecture: search over harness changes, evaluate candidates, and protect the held-out test.

Bring one proposed harness change and the experiment that could disprove its benefit.